

编译系统实验二报告：语义分析

姓名：顾雨航 学号：2023112451

一、程序实现的功能

本实验在实验一词法分析和语法分析的基础上，实现了 C-- 语言的语义分析阶段。程序在语法树构造成功后遍历抽象语法树，建立变量、函数和结构体符号表，并完成必做部分要求的 17 类语义错误检查。对于没有语义错误的程序不输出额外信息；如果发现错误，则按照实验指导书要求输出 Error type、行号和简短说明。

- 变量和函数相关检查：支持变量未定义、函数未定义、变量重复定义、函数重复定义、普通变量被当作函数调用、函数实参与形参数量或类型不匹配等错误。
- 类型相关检查：支持赋值左右类型不匹配、赋值左边不是左值、运算符操作数类型不匹配、return 表达式类型与函数返回类型不匹配等错误。
- 数组和结构体相关检查：支持非数组使用下标、数组下标不是整数、非结构体使用点号、访问不存在的结构体域、结构体域重复定义或初始化、结构体名重复、使用未定义结构体定义变量等错误。

二、主要方法

语义分析入口位于 lab2_code/main.c。主程序在词法、语法分析没有报错时调用 semantic_analyze(root)，把语法树根节点交给 semantic.c 处理。这样实验一与实验二的职责比较清楚：实验一负责得到正确的语法树，实验二只基于语法树做名字检查和类型检查。

核心实现集中在 lab2_code/semantic.c 和 lab2_code/semantic.h。semantic.h 定义语义分析模块对外接口和类型结构；semantic.c 负责符号表维护、类型构造、语法树遍历和错误报告。类型系统使用统一的 Type 结构表示 int、float、数组和结构体，其中数组类型通过递归的元素类型表示，结构体类型通过域链表保存成员信息。

程序维护变量表、函数表和结构体表三类符号表。变量表记录变量名、类型和定义行号；函数表记录函数名、返回类型和形参列表；结构体表记录结构体名及其域列表。由于必做部分假设变量、函数和结构体都处于全局作用域，插入符号时直接在全局表中检查重名关系。

语法树遍历采用自顶向下的方式：ExtDefList 和 ExtDef 处理全局定义；Specifier 构造类型；FunDec 和 VarList 处理函数名与形参；VarDec 处理普通变量和数组变量；CompSt、DefList、StmtList 和 Stmt 处理函数体、局部定义、语句和 return。表达式检查集中在 analyze_exp 中完成，它根据表达式产生式判断变量、函数调用、数组访问、结构体域访问、赋值和各种运算，并返回该表达式的类型。

类型等价判断集中在 same_type 中完成，错误输出集中在 report_error 中完成。这样赋值、return、函数传参、数组元素访问等场景可以复用同一套类型比较规则，避免不同位置的判断标准不一致。

三、典型错误处理方法

典型错误处理主要集中在类型比较、表达式分析和函数实参检查三个位置。

- 类型不匹配类错误：赋值、return 和函数传参都需要判断两个 Type 是否等价，因此程序把基本类型、数组类型和结构体类型的比较集中放在 same_type 中。

```
static int same_type(Type *left, Type *right) {
    if (left == NULL || right == NULL) {
        return 1;
    }
    if (left->kind != right->kind) {
        return 0;
    }

    switch (left->kind) {
        case TYPE_BASIC:
            return left->u.basic == right->u.basic;
        case TYPE_ARRAY:
            return same_type(left->u.array.elem, right->u.array.elem);
        case TYPE_STRUCT:
            if (left->u.structure.name == NULL || right->u.structure.name == NULL) {
                return 1;
            }
            return strcmp(left->u.structure.name, right->u.structure.name) == 0;
    }
}
```

- 表达式类错误：变量未定义、函数未定义、变量当函数、数组访问、结构体访问、赋值左值和普通运算类型错误都在 analyze_exp 中根据表达式产生式分类处理。

```
if (is_node(first, "ID") && second == NULL) {
    VarSymbol *var = find_variable(first->value.text);
    if (var == NULL) {
        report_error(1, first->line, "Undefined variable \"%s\"", first->value.text);
        return NULL;
    }
    return copy_type(var->type);
}
```

- 函数调用类错误：函数调用时，程序把每个实参表达式的类型与函数表中保存的形参类型逐个比较，并检查参数数量是否一致。

```
static int analyze_args(TreeNode node, FuncSymbol *func) {
    Field *formal = func->params;
    TreeNode cur = node;
    int mismatch = 0;

    while (!is_empty(cur)) {
        Type *actual = analyze_exp(child(cur, 0));
        if (formal == NULL) {
            mismatch = 1;
        } else if (!same_type(actual, formal->type)) {
            mismatch = 1;
        } else {
            formal = formal->next;
        }
        free_type(actual);

        if (child(cur, 1) == NULL) {
            break;
        }
        cur = child(cur, 2);
    }

    if (formal != NULL) {
        mismatch = 1;
    }
}
```

此外，重复定义类错误主要发生在插入符号表之前，例如 `add_variable`、`add_function` 和 `add_structure` 会先查询现有符号，再决定是否报错。

四、编译与运行方法

进入实验二代码目录后可以直接使用 `Makefile` 编译。

```
make
```

编译成功后会生成可执行文件 `parser`。对单个测试样例运行时，输入文件名作为命令行参数：

```
./parser testcases/required_01
```

在 Windows 的 PowerShell 或 `cmd` 环境中，如果使用 MinGW/MSYS2 等工具链，运行方式可写为：

```
.\parser.exe testcases\required_01
```

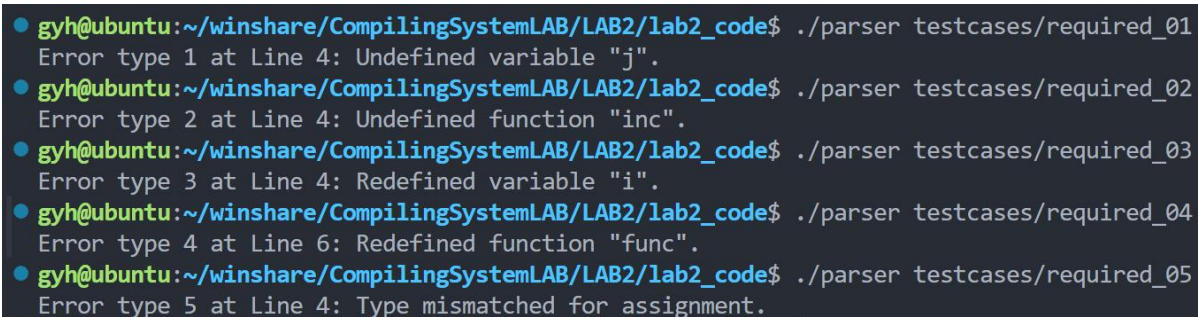
如果需要一次运行所有自带必做测试样例，可以执行：

```
make test
```

编译依赖包括 Flex、Bison、GCC 和 Make。

五、测试结果

本实验准备了 `required_01` 到 `required_17` 共 17 个必做测试样例，分别覆盖指导书列出的 17 类语义错误。运行结果应输出对应的 Error type 和行号。下图为部分输出结果：



```
gyh@ubuntu:~/winshare/CompilingSystemLAB/LAB2/lab2_code$ ./parser testcases/required_01
Error type 1 at Line 4: Undefined variable "j".
gyh@ubuntu:~/winshare/CompilingSystemLAB/LAB2/lab2_code$ ./parser testcases/required_02
Error type 2 at Line 4: Undefined function "inc".
gyh@ubuntu:~/winshare/CompilingSystemLAB/LAB2/lab2_code$ ./parser testcases/required_03
Error type 3 at Line 4: Redefined variable "i".
gyh@ubuntu:~/winshare/CompilingSystemLAB/LAB2/lab2_code$ ./parser testcases/required_04
Error type 4 at Line 6: Redefined function "func".
gyh@ubuntu:~/winshare/CompilingSystemLAB/LAB2/lab2_code$ ./parser testcases/required_05
Error type 5 at Line 4: Type mismatched for assignment.
```

六、实验总结

本次实验的重点是从语法树进一步抽象出类型信息和符号信息，并利用这些信息检查程序中的语义错误。实现过程中，最关键的是把符号表维护、类型构造和表达式类型推导分清楚：符号表解决名字是否存在以及是否重复的问题，类型系统解决不同表达式之间能否匹配的问题，语法树遍历则把两者串联起来。通过独立的 `semantic` 模块，本程序较清晰地完成了必做部分要求，也方便继续扩展更复杂的作用域或选做语义检查。